DESENVOLVIMENTO BACK-END · UNIDADE V

Autenticação, Autorização e Segurança

Protegendo a API acadêmica em camadas

OWASP · JWT · RBAC · Hash · CORS · Rate limiting · Logs

🎯 Objetivos de aprendizagem

🛡️

Identificar

as principais vulnerabilidades do OWASP Top 10:2025

🪪

Diferenciar

autenticação de autorização, e 401 de 403

🎟️

Explicar

sessões, JWT, refresh tokens e MFA

👥

Aplicar

o controle de acesso por papéis (RBAC) com policies

🔐

Distinguir

hash de criptografia no armazenamento de dados

🧱

Relacionar

CORS, rate limiting e logs à defesa em camadas

💡 Basta trocar um número na URL?

Uma auditoria descobriu que, alterando o id em `/api/v1/matriculas/{id}`, um aluno **via as notas dos colegas**.

- O login aceitava **tentativas ilimitadas**
- Os logs **não registravam** quem acessou o quê
- O usuário estava **autenticado**, mas ninguém verificou se **podia** ver aquele dado

Analogia

Ter o **crachá** da empresa não significa poder entrar na **sala da diretoria**. Cada porta precisa da sua própria verificação.

Segurança não é uma etapa final: é uma **qualidade construída** ao longo de todo o desenvolvimento.

🛡️ OWASP Top 10:2025

Código	Categoria
A01	Controle de acesso quebrado
A02	Configuração insegura
A03	Cadeia de suprimentos
A04	Falhas criptográficas
A05	Injeção

Código	Categoria
A06	Design inseguro
A07	Falhas de autenticação
A08	Falhas de integridade
A09	Falhas de log e alerta
A10	Tratamento inadequado de exceções

📦 **A03 na prática:** rode `composer audit` no pipeline, versione o `composer.lock` e use o Dependabot para receber correções de segurança.

💉 Injeção de SQL

```php
// ❌ VULNERÁVEL: o valor do usuário é concatenado ao SQL
$usuario = DB::select("SELECT * FROM users WHERE email = '$email'");
// Com o valor:  qualquer' OR '1'='1   → a consulta devolve TODOS os usuários

// ✅ SEGURO: consultas parametrizadas
$usuario = DB::select('SELECT * FROM users WHERE email = ?', [$email]);
$usuario = User::where('email', $email)->first();        // ORM

// ⚠ Cuidado com métodos "raw": use sempre os parâmetros
User::whereRaw('lower(email) = ?', [$email]);
```

📌 **O que demonstra:** dados do usuário **nunca** devem ser concatenados a comandos. Escapar caracteres funciona, mas depende de nunca esquecer: a OWASP recomenda **consultas parametrizadas**.

🧪 XSS, exposição e atribuição em massa

📜

XSS

Um `<script>` salvo numa "observação" roda no navegador de **outro usuário**.

Defesa: escapar a saída (`{{ }}` no Blade) e responder sempre JSON

🔓

Exposição de dados

Devolver o model `User` inteiro pode vazar o **hash da senha** e tokens.

Defesa: `$hidden` no model e **API Resources** com campos explícitos

📥

Atribuição em massa

`User::create($request->all())` aceita até um campo de **papel de administrador**.

Defesa: `$fillable` e `$request->validated()`

Princípio comum: a API só **aceita** e só **devolve** o que foi **declarado explicitamente**.

🚪 Controle de acesso quebrado (IDOR)

```php
// ❌ VULNERÁVEL: autenticado ≠ autorizado. GET /matriculas/1, /2, /3...
public function show(Matricula $matricula): MatriculaResource
{
    return new MatriculaResource($matricula);
}

// ✅ Policy: concentra a regra "quem pode ver esta matrícula?"
public function view(User $user, Matricula $matricula): bool
{
    return $user->hasRole('ADMIN')
        || $matricula->aluno->user_id === $user->id                    // dono
        || $matricula->disciplina->professor?->user_id === $user->id;  // professor da turma
}

// ✅ Controller corrigido: 403 Forbidden se a policy negar
Gate::authorize('view', $matricula);
```

📌 **O que demonstra:** toda rota que devolve ou altera dados precisa responder **"este usuário pode acessar ESTE objeto?"**. UUIDs dificultam adivinhar ids, mas **não substituem** a verificação.

🍪 CSRF e cabeçalhos de segurança

CSRF

- O navegador envia **cookies automaticamente**
- Um site malicioso pode disparar uma ação **em nome do professor**
- **Defesas:** cookie `SameSite` e **token CSRF**
- APIs com `Authorization: Bearer` **não sofrem** o CSRF clássico

Cabeçalho	Protege contra
`X-Content-Type-Options`	executar arquivos como script
`X-Frame-Options`	clickjacking
`Content-Security-Policy`	recursos de origens indevidas
`Strict-Transport-Security`	acesso sem HTTPS

💡 Um **middleware** adiciona esses cabeçalhos a **todas** as respostas, sem que nenhum controller precise se preocupar.

Autenticação × Autorização

Aspecto	Autenticação	Autorização
Pergunta	**quem é você?**	**o que você pode fazer?**
Momento	primeiro	depois
Mecanismos	senha, token, MFA	papéis, policies
Falha gera	**401** Unauthorized	**403** Forbidden
Exemplo	login com e-mail e senha	só professores lançam notas

Analogia

Na portaria da empresa, o **crachá** prova quem você é. A **catraca** de cada andar decide se você pode entrar ali.

Primeiro **identificar**, depois **permitir**: não existe autorização sem autenticação.

🎟️ Sessão × JWT

Critério	Sessão com cookie	JWT (Bearer)
Estado no servidor	**sim** (stateful)	**não**, para o access token (stateless)
Revogação imediata	simples: apagar a sessão	difícil: expiração curta ou lista de bloqueio
Escalar em vários servidores	exige armazenamento compartilhado (Redis)	natural
Clientes ideais	portal web no mesmo domínio	apps móveis, SPAs, integrações
Principal risco	CSRF	roubo do token

🧭 Não há abordagem superior em todos os cenários. Na API acadêmica, consumida por **apps e sistemas parceiros**, usamos **JWT com refresh token**. No Laravel, o **Sanctum** oferece as duas opções.

🧩 Anatomia de um JWT

Header
algoritmo: HS256

.

Payload
sub, roles, iat, exp

.

Assinatura
header + payload + chave

✍️

Integridade

Trocou `ALUNO` por `ADMIN` no payload? A assinatura **não confere** e o token é rejeitado.

👀

Não é segredo

O payload é só **codificado** em Base64URL: **qualquer pessoa** consegue ler.

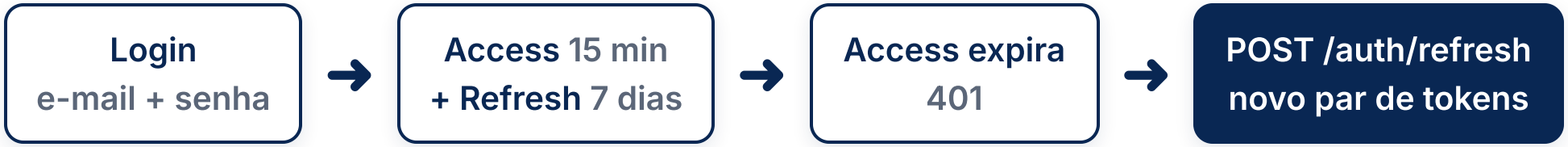
⏳

Vida curta

Access token de **~15 minutos**, sempre com a claim `exp`.

⚠️ Nunca coloque **senhas ou dados sensíveis** no payload. No servidor, aceite **apenas o algoritmo esperado**: bibliotecas antigas já aceitaram tokens com `alg: none`.

🔄 Access token + refresh token



- Refresh token **guardado no servidor**, só o **hash** (SHA-256)
- **Rotação:** cada uso invalida o anterior
- **Reuso** de um token antigo = **roubo** → revogar todos

🔒 No login, mensagem **genérica**: "Credenciais inválidas." Dizer "e-mail não encontrado" ajuda o atacante a descobrir contas.

🔑 Senhas fortes e autenticação multifator

Fator	O que é	Exemplos
Conhecimento	algo que você **sabe**	senha, PIN
Posse	algo que você **tem**	app autenticador (TOTP), chave física
Inerência	algo que você **é**	digital, reconhecimento facial

Recomendações atuais (NIST)

Senhas **longas** em vez de regras complexas, checagem contra **senhas vazadas** e nada de troca periódica sem motivo.

💡 No Laravel: `Password::min(12)→uncompromised()`. Para **administradores e professores**, exigir o **segundo fator** no login.

👥 Autorização com RBAC

Usuário
Profa. Ana

→

Papel
PROFESSOR

→

Permissões
notas.lancar · disciplinas.ver

Ação	ADMIN	PROFESSOR	ALUNO
Gerenciar cursos e usuários	sim	não	não
Lançar notas	sim	**só nas suas turmas**	não
Ver notas	todas	das suas turmas	**só as próprias**

🧭 **Middleware de papel** responde "é professor?"; **policies e gates** respondem "é professor **desta** turma?". **Menor privilégio:** negue por padrão e verifique sempre no back-end.

🔐 Hash × criptografia

Critério	Hash	Criptografia
Reversível?	**não** (mão única)	**sim**, com a chave
Uso típico	**senhas**, integridade	dados que precisam ser **lidos** (documento, telefone)
Algoritmos	bcrypt, Argon2id · SHA-256	AES-256 (simétrica), RSA
No Laravel	`Hash::make()`, `Hash::check()`	cast `encrypted`, `Crypt::encryptString()`

🐢 **Lento de propósito:** bcrypt custo 10 ≈ **61 ms**; SHA-256 ≈ **0,0004 ms**. Para o usuário, imperceptível; para a força bruta, **inviável**.

🧂 **Salt** automático: senhas iguais geram hashes **diferentes**. A chave `APP_KEY` fica **fora** do repositório.

🚦 CORS, rate limiting e logs

🌐

CORS

Diz ao navegador **quais origens** podem chamar a API. Liste as origens confiáveis; nunca `'*'` em produção.

Não é autenticação: o curl ignora.

⏱️

Rate limiting

Limita requisições por tempo. Excedeu? **429 Too Many Requests**. No login: chave por **e-mail + IP**.

Vários servidores? Contadores no Redis.

📋

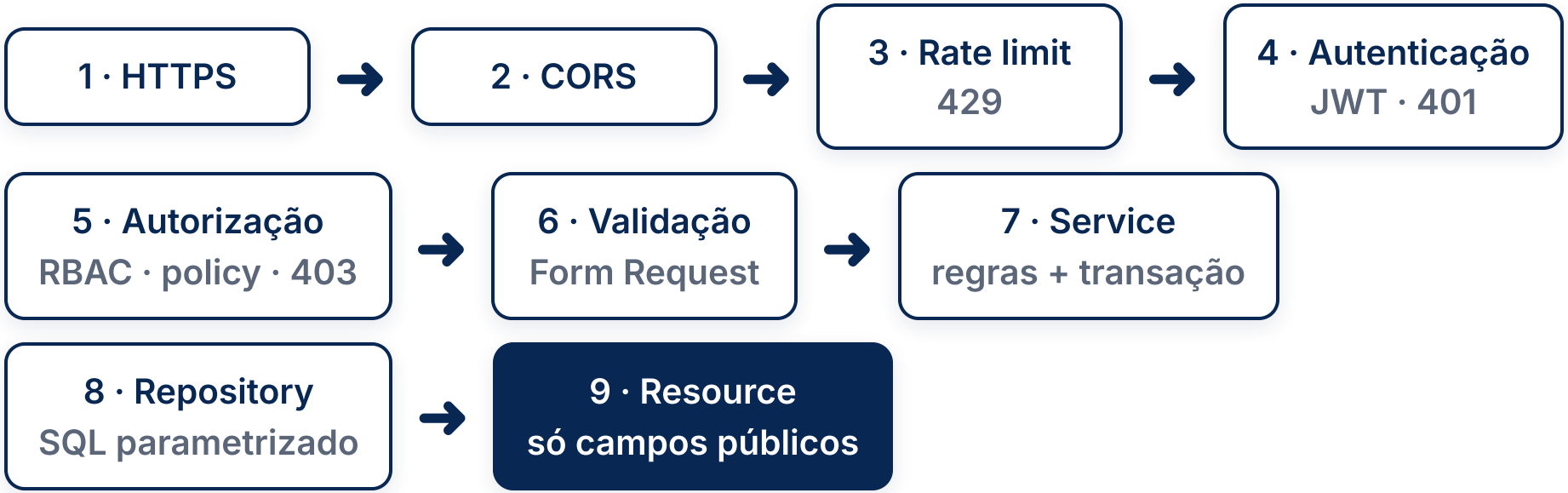
Logs e auditoria

Auditoria responde **quem fez o quê, quando e de onde**. Monitore picos de **401, 403 e 429**.

Logs só valem se alguém olhar.

⚠️ **Nunca** registre senhas, tokens ou documentos completos nos logs: um log com dados sensíveis é um **vazamento esperando para acontecer**.

🧱 Defesa em profundidade



Nenhuma camada basta sozinha: se uma **falhar**, a próxima ainda **impede o ataque**. Logs e tratamento de exceções acompanham todo o fluxo.

🧭 Segurança no dia a dia: analogias

🍊 Hash · Mão única

Do **suco** não dá para voltar à laranja. Mas dá para espremer outra e comparar o sabor.

Laravel: `Hash::check()`

🗝️ Criptografia · Reversível

Um **cofre**: quem tem a chave abre e lê o conteúdo de volta.

Laravel: cast `encrypted` + `APP_KEY`

🏧 Rate limiting

O **caixa eletrônico** bloqueia o cartão depois de três senhas erradas.

Laravel: `throttle:login` → 429

🏰 Defesa em profundidade

O **castelo** tem fosso, muralha, portão e guardas: vencer um obstáculo não basta.

HTTPS → CORS → auth → policy → validação

💡 Pergunta-chave: **se eu fosse o atacante, por onde tentaria entrar, e qual camada me impediria?**

🧾 Síntese dos conceitos

- **OWASP Top 10**: checklist das falhas mais críticas
- **Consultas parametrizadas** contra injeção
- Só aceitar e devolver o que é **declarado**
- **401** = quem é você? · **403** = você pode?
- **JWT curto** + **refresh** com rotação
- **RBAC + policies**: papel e propriedade do recurso
- **Hash** para senhas, **criptografia** para dados legíveis
- **CORS, rate limiting e logs** completam a defesa

🧭 Segurança não é um recurso que se instala no final: é a soma de **pequenas práticas consistentes** em **todas as camadas**.

📝 Reflexão

```php
// Desafio: encontre ao menos 5 falhas de segurança
// e indique a camada que corrigiria cada uma
Route::get('/v1/matriculas/{id}', function ($id) {
    $m = DB::select("SELECT * FROM matriculas WHERE id = $id");
    Log::info('consulta', ['senha' => request('senha')]);
    return $m;
});
```

💬 Reflita

1. Por que esconder um botão no front-end **não** é controle de acesso?
2. Quando você escolheria **sessão** em vez de **JWT**?
3. Por que um hash **rápido** é ruim para senhas?